Linux : Appels Systèmes - BINV2181-B (fork et exec)

SOLUTIONS

2.A. Exercice sur l'appel système : `fork`

a) Ecrivez un programme qui définit une variable « a » de type `int` et l'initialise à 5, crée un processus fils, affiche la valeur de retour de l'appel système `fork` . Ensuite, il définit dans le fils une nouvelle variable « b » de type `int`, lui affecte la valeur de « a * 2 » puis affiche le contenu de a et de b ; le père définit une autre variable « b » de type int, lui affecte la valeur de « a * 5 » puis affiche le contenu de a et de b. Pour cet exercice, l'affichage peut se faire avec `printf`.

Qu'affiche ce programme lors de son exécution ?

```
la variable pidFils vaut : 9322
dans le pere : a = 5, b = 25
la variable pidFils vaut : 0
dans le fils : a = 5, b = 10
```

ou

```
la variable pidFils vaut : 9327
la variable pidFils vaut : 0
dans le fils : a = 5, b = 10
dans le pere : a = 5, b = 25
```

b) Ecrivez un programme qui affiche « trois .. deux .. un .. » avant de créer un fils qui affiche « partez ! ». Les écritures doivent se faire grâce à l'appel système `write`.

Qu'affiche ce programme lors de son exécution ?

```
trois .. deux .. un .. partez!
```

c) Ajoutez '\n' à la fin des deux chaînes de caractères.

Qu'affiche ce programme modifié lors de son exécution ?

```
trois .. deux .. un ..
partez!
```

d) Refaites les étapes b et c en remplaçant les appels système `write` par l'appel à la fonction `printf`.

Qu'affiche le programme sans '\n' lors de son exécution ?

```
trois .. deux .. un .. trois .. deux .. un .. partez!
```

Qu'affiche ce programme, si les chaînes de caractères se terminent par '\n'?

```
trois .. deux .. un ..
partez!
```

e) Y-a-t-il une différence entre les deux versions ? Si oui, justifiez-la puis neutralisez-la.

Oui. La sortie standard étant un « character device », le buffer du *printf* n'est vidé que lorsqu'il est plein ou s'il contient un '\n'. Sans '\n', il n'est pas vidé et le fils hérite d'un buffer qui contient déjà « `trois .. deux .. un ..` ».
A la fin du programme, les buffers sont vidés : celui du père qui contient « `trois .. deux .. un ..` » et celui du fils qui contient également « `trois .. deux .. un .. partez!` »

> RAPPEL : Les buffers sont vidés :
>
> - quand le stream associé au buffer est fermé → p.ex. le processus se termine
> - si le buffer est *line buffered*, quand il rencontre une newline → c'est le cas des *character devices* (ex : clavier/écran)
> - si le buffer est *full buffered*, quand il est plein → c'est le cas des *character devices* & des *block devices* (ex : fichier sur disque)
> - s'il s'agit d'un *output buffer*, lorsqu'il est lu → p.ex. appel système `read()`

Pour résoudre le problème, seul **`fflush(stdout)`** est valable puisque l'affichage est opéré grâce à *printf()* ; le buffer à vider est donc celui qui est associé au stream *stdout*.

Note 1 : **`tcflush(1,TCOFLUSH)`** ne convient pas car il vide le buffer de la sortie standard, supprimant toutes les données en attente d'être affichées. Ces données étant effacées, elles seront perdues et ne pourront pas être affichées à l'écran.

Note 2 : contrairement à la sortie standard *stdout*, la sortie d'erreur *stderr* n'est pas bufferisée pour garantir que les messages d'erreur soient affichés immédiatement. Par conséquent, la fonction **`fprintf(stderr,...)`** ne présente pas le problème d'affichage précédent lié à la bufferisation des écritures.

f) En conclusion, quelle est la différence majeure entre l'appel système `write` et la fonction `printf` ?

L'appel système *write* n'est pas bufferisé alors que la fonction *printf* l'est (ce qui rend cette dernière beaucoup plus rapide que le syscall *write*).

2.B. Exercice sur les appels système : fork et wait

a) Ecrivez un programme qui crée un fils qui affiche « 4 5 6\n » alors que le père affiche « 1 2 3\n ».
Testez ce programme et expliquez le résultat obtenu.

```
1 2 3
4 5 6
```

Le père et le fils s'exécutent « en parallèle » (ou disons plutôt comme un bon OS multi tâches où le *scheduler* (ordonnanceur) alloue le temps CPU entre les processus en cours).

Ici le papa affiche « 1 2 3\n » puis se termine et le fils affiche « 4 5 6\n » et se termine.